

PORTADA

Técnicas de Programación

Unidad 4 — Librerías para Análisis y Visualización de Datos

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Cerca de 1 de cada 12 hombres tiene algún grado de daltonismo — un gráfico que solo se entiende "por el color" ya excluyó a una parte real de quien lo lee.



TRANSICIÓN

Cerrando la Unidad 4

Ya sabes leer y escribir **archivos de texto, CSV y JSON**, manejar **excepciones** cuando algo sale mal, generar **reportes en PDF** y verificar tu código con **JUnit**. Hoy das el siguiente paso: usar librerías especializadas para **analizar** datos (en vez de parsear CSV a mano) y **visualizarlos** en gráficos — y cerrar con un tema transversal a todo lo anterior: diseñar pensando en que **cualquier persona** pueda usar lo que construyes.

CONCEPTO

Apache Commons CSV: por qué no usar `split(",")`

En la sesión de archivos viste cómo leer un CSV partiendo cada línea con `split(",")` . Funciona en casos simples, pero se rompe fácil:

- Un valor con una **coma dentro** (ej. "Bogotá, Colombia") se parte donde no debía.
- Valores entre **comillas** no se interpretan como un solo campo.
- No hay soporte incorporado para leer la primera fila como **encabezados** y acceder por nombre de columna.

Apache Commons CSV es una librería que resuelve exactamente esos tres problemas — maneja comillas, comas dentro de valores y encabezados de forma correcta y probada, en vez de reinventar ese parseo a mano.

CONCEPTO

Leer un CSV con CSVParser

Un `CSVParser` lee el archivo y lo entrega como una secuencia de `CSVRecord` — cada uno representa una fila, y permite acceder a sus columnas por nombre con `record.get("columna")` cuando el archivo tiene encabezados.

```
Reader lector = new FileReader("estudiantes.csv");
CSVFormat formato = CSVFormat.DEFAULT.builder()
    .setHeader().setSkipHeaderRecord(true).build();

try (CSVParser parser = new CSVParser(lector, formato)) {
    for (CSVRecord record : parser) {
        String nombre = record.get("nombre");
        double nota = Double.parseDouble(record.get("nota"));
        System.out.println(nombre + ": " + nota);
    }
}
```

CONCEPTO

Apache Commons Math: estadística sin reinventarla

Apache Commons Math es una librería de utilidades matemáticas y estadísticas para Java. En vez de calcular a mano el promedio, sumando y dividiendo, o programar tu propia fórmula de desviación estándar, la clase `DescriptiveStatistics` lo hace por ti a partir de una colección de valores.

- `addValue(double)` : agrega un dato a la colección.
- `getMean()` : retorna el promedio.
- `getStandardDeviation()` : retorna la desviación estándar.
- `getPercentile(50)` : retorna la mediana (percentil 50).

CONCEPTO

DescriptiveStatistics en código

```
DescriptiveStatistics stats = new DescriptiveStatistics();

for (CSVRecord record : parser) {
    stats.addValue(Double.parseDouble(record.get("nota")));
}

double promedio = stats.getMean();
double desviacion = stats.getStandardDeviation();
double mediana = stats.getPercentile(50);

System.out.printf("Promedio: %.2f, Mediana: %.2f, Desv.Std: %.2f%n",
    promedio, mediana, desviacion);
```

Nota cómo se combina con lo de la diapositiva anterior: los valores llegan del `CSVParser` , y `DescriptiveStatistics` los resume.

TRANSICIÓN

De los números al gráfico

Tener el promedio o la mediana en la consola es útil, pero un gráfico comunica una tendencia mucho más rápido que una tabla de números. Para eso existe **JFreeChart**, una librería Java para crear gráficos de barras, líneas, torta y más, y exportarlos como imagen.

CONCEPTO

JFreeChart: agregar la dependencia y el dataset

Como con JUnit y Commons CSV/Math, JFreeChart se agrega como dependencia de Maven:

```
<dependency>
  <groupId>org.jfree</groupId>
  <artifactId>jfreechart</artifactId>
  <version>1.5.4</version>
</dependency>
```

Antes de crear el gráfico, los datos se organizan en un `DefaultCategoryDataset` — una estructura que agrupa valores por **serie** (qué se está midiendo) y **categoría** (el eje X):

```
DefaultCategoryDataset dataset = new DefaultCategoryDataset();
dataset.addValue(promedio, "Notas", "Grupo A");
dataset.addValue(3.8, "Notas", "Grupo B");
```

CONCEPTO

Crear y exportar el gráfico

`ChartFactory` construye el gráfico a partir del dataset, y `ChartUtils` lo exporta como imagen PNG a un archivo:

```
JFreeChart grafico = ChartFactory.createBarChart(  
    "Promedio de notas por grupo", // título  
    "Grupo",                       // eje X  
    "Nota",                       // eje Y  
    dataset  
);  
  
ChartUtils.saveChartAsPNG(  
    new File("promedios.png"), grafico, 800, 600  
);
```

`ChartFactory.createLineChart(...)` tiene la misma firma y sirve para mostrar una tendencia en vez de comparar categorías — se elige el tipo de gráfico según qué pregunta se quiere responder.

CONCEPTO

El flujo completo, de punta a punta

Las tres librerías de hoy se encadenan en un solo flujo típico de análisis de datos:

1. **Leer** el CSV con `CSVParser` / `CSVRecord` (Apache Commons CSV).
2. **Calcular** estadísticas descriptivas con `DescriptiveStatistics` (Apache Commons Math).
3. **Graficar** el resultado con `DefaultCategoryDataset` + `ChartFactory` (JFreeChart).
4. **Exportar** el gráfico a PNG con `ChartUtils.saveChartAsPNG` para incluirlo en un reporte o PDF.

Es el mismo patrón que ya usaste al generar PDFs: cada librería resuelve un paso específico, bien probado, en vez de que tu código reimplemente el parseo, la estadística o el dibujo del gráfico desde cero.

INTERACCIÓN

Elijan el gráfico correcto

En parejas, respondan: tienen las notas de un grupo registradas cada semana durante el semestre, y quieren mostrar cómo cambia el promedio del grupo a lo largo del tiempo. ¿Usarían `createBarChart` o `createLineChart` ? Justifiquen con una frase.

5 minutos · piensen en qué pregunta responde cada tipo de gráfico

TRANSICIÓN

¿Y quién puede leer ese gráfico?

Ya sabes construir un gráfico técnicamente correcto. Falta una pregunta igual de importante: ¿ese gráfico lo puede leer **cualquier persona** que lo reciba, incluyendo alguien con discapacidad visual, motora o cognitiva? Esa pregunta es el corazón del **software inclusivo**.

CONCEPTO

Diseño universal / software inclusivo

El **diseño universal** (también llamado **software inclusivo**) es el enfoque de diseñar productos de software para que la **mayor cantidad posible de personas** puedan usarlos, sin necesitar una versión adaptada aparte — incluyendo personas con discapacidad visual, motora o cognitiva, y también a cualquiera en una situación temporal (una pantalla con mucho brillo de sol, un brazo lesionado).

No es un tema exclusivo de aplicaciones web o móviles: aplica igual a una app de escritorio, a un reporte en PDF, o a los gráficos que generaste hace un momento con JFreeChart.

CONCEPTO

Principios aplicados a gráficos y reportes

Principio	Por qué importa
No depender solo del color	Usa también patrones, formas o etiquetas de texto directamente sobre las barras/líneas — así la información no se pierde para quien no distingue esos colores.
Paletas de buen contraste	Colores muy cercanos en tono (o muy claros sobre fondo claro) son difíciles de distinguir para baja visión, y se ven peor al imprimir en blanco y negro.
Texto alternativo / descripciones	Un gráfico debe venir acompañado de una descripción corta de lo que muestra, útil para lectores de pantalla y como resumen rápido para cualquiera.
Tamaños de fuente legibles	Etiquetas de ejes y leyendas muy pequeñas excluyen a quien tiene baja visión, sin necesitar zoom.

CONCEPTO

WCAG como referencia estándar

Las **WCAG** (*Web Content Accessibility Guidelines*, Pautas de Accesibilidad para el Contenido Web) son el estándar más reconocido de la industria para diseño accesible, publicado por el W3C. Aunque su nombre menciona "web", sus criterios sobre contraste de color, tamaños de texto y no depender solo del color son un buen punto de referencia incluso para aplicaciones de escritorio o reportes impresos, como los que construyes con JFreeChart.

Usarlas como referencia no significa que una app de escritorio deba "cumplir WCAG" formalmente — significa aprovechar criterios ya probados en vez de inventar los propios.

Lo que hay que llevarse de la sesión y de la unidad

1. Apache Commons CSV parsea archivos CSV correctamente (comillas, comas internas, encabezados) donde `split(",")` falla.
2. Apache Commons Math (`DescriptiveStatistics`) calcula promedio, desviación estándar y mediana sin programar las fórmulas a mano.
3. JFreeChart construye gráficos de barras/líneas a partir de un `DefaultCategoryDataset` y los exporta a PNG con `ChartUtils` .
4. El software inclusivo diseña para que la mayor cantidad de personas posible pueda usar lo que construyes — color + patrón, buen contraste, texto alternativo, fuentes legibles, con WCAG como referencia.
5. **Resumen de toda la Unidad 4:** excepciones (manejar errores), archivos de texto/CSV/JSON y PDF (persistir y reportar datos), JUnit (verificar que el código funciona) y las librerías de hoy (analizar, graficar y presentar esos datos de forma inclusiva).



CIERRE

Antes del Laboratorio 2

- Repasa las seis sesiones de la Unidad 4 (excepciones, archivos/CSV/JSON, PDF, JUnit, análisis y visualización de datos, software inclusivo) — son la base directa del **Laboratorio 2**.
- Ten instalado y probado Maven con las dependencias vistas hoy (Commons CSV, Commons Math, JFreeChart) antes de la sesión del laboratorio.

Próxima sesión: Laboratorio 2 — aplicación integradora de la Unidad 4.

